Network and System Management

Level 4: Step 1 - Not so S(imple)NMP

Student Name: Hasib Rahman
Student ID: a57540 – (Hasib Rahman) & a61521 – (Md Nahid Hasna Refat)

Question 1: FCAPS Areas Interaction in Modern Network Management

FCAPS stands for Fault, Configuration, Accounting, Performance, and Security - the five functional areas defined by ISO for network management. In modern multi-layer network architectures, these areas don't work in isolation; they constantly interact and share information to provide comprehensive network oversight.

How FCAPS Areas Interact:

In a typical enterprise network, the Configuration management system maintains a database of all network devices and their settings. When the Fault management system detects an issue, it queries this configuration database to understand the affected device's role and dependencies. Similarly, Performance management continuously monitors metrics like bandwidth utilization and latency, feeding this data to both Fault management (to detect anomalies) and Accounting management (for usage billing).

Security management ties everything together by ensuring that all management operations are authenticated and authorized, while also monitoring for security-related faults.

Real Scenario: Fault + Performance Overlap

A practical example of FCAPS overlap occurs in network congestion detection. Consider a core router in a data center experiencing high CPU utilization. The Performance management system tracks CPU usage metrics via SNMP polling (using OIDs from HOST-RESOURCES-MIB). When utilization exceeds 85% for a sustained period, this triggers a threshold violation.

This same event is simultaneously handled by Fault management, which generates an alert and potentially a trap notification to the network operations center. The fault system correlates this with other performance data - perhaps increased packet drops or latency spikes on the same device - to determine if this represents a genuine fault condition requiring intervention.

In my practical implementation, I configured exactly this scenario: a monitoring script that tracks CPU usage (Performance) and sends SNMP traps when thresholds are exceeded (Fault), demonstrating how these two FCAPS areas work together in real-time.

Question 2: Comparison of SMIv1, SMIv2, and SMIv2-Experimental

Differences in Data Types:

SMIv1 introduced basic data types including INTEGER, OCTET STRING, OBJECT IDENTIFIER, NULL, IpAddress, Counter, Gauge, TimeTicks, and Opaque. These were sufficient for early network management but had limitations.

SMIv2 expanded the type system significantly. It added Counter64 for high-speed interfaces where 32-bit counters would wrap too quickly, Integer32 and Unsigned32 for clearer semantics, and BITS for representing bit flags. SMIv2 also introduced textual conventions through the SNMPv2-TC module, allowing more meaningful type definitions like DisplayString, TruthValue, and DateAndTime.

SMIv2-Experimental was a transitional specification that appeared during the development of SNMPv2. It contained experimental features that were being tested before standardization. Some features were adopted into the final SMIv2, while others were dropped.

Structure of OBJECT-TYPE Definitions:

In SMIv1, an OBJECT-TYPE definition uses this structure:

```
objectName OBJECT-TYPE
  SYNTAX     Integer32
  ACCESS     read-only
  STATUS     mandatory
  DESCRIPTION "Description text"
  ::= { parentObject 1 }
```

SMIv2 modified this structure with important changes:

```
objectName OBJECT-TYPE
  SYNTAX     Integer32
  MAX-ACCESS  read-only
  STATUS     current
  DESCRIPTION "Description text"
  ::= { parentObject 1 }
```

The key differences are: ACCESS became MAX-ACCESS (indicating the maximum access level an implementation might provide), and STATUS values changed from "mandatory/optional/obsolete" to "current/deprecated/obsolete". SMIv2 also added UNITS and DEFVAL clauses for better documentation.

Why Vendors Still Expose SMIv1-Compatible MIBs:

Several practical reasons explain the persistence of SMIv1 MIBs. First, backward compatibility remains crucial - many organizations still operate legacy NMS platforms that only understand SMIv1. Second, the transition cost is significant; rewriting and testing MIBs requires resources that vendors may not prioritize. Third, SMIv1 MIBs continue to function adequately for many use cases, so there's no urgent need to migrate. Finally, some embedded devices with limited resources find SMIv1's simpler structure easier to implement.

Question 3: IF-MIB Research Using simpleweb.org

(a) OID for ifHighSpeed:

The OID for ifHighSpeed in the IF-MIB is:

1.3.6.1.2.1.31.1.1.1.15

This object represents the interface speed in units of 1,000,000 bits per second. It was introduced because the original ifSpeed object (a Gauge32) couldn't represent speeds above approximately 4.3 Gbps.

(b) Objects in IF-MIB Using SYNTAX Gauge32:

The following objects in IF-MIB use Gauge32 as their SYNTAX:

- **ifSpeed** (1.3.6.1.2.1.2.2.1.5) - Interface bandwidth estimate in bits per second
- **ifInUnknownProtos** (1.3.6.1.2.1.2.2.1.15) - Inbound packets with unknown protocol
- **ifOutQLen** (1.3.6.1.2.1.2.2.1.21) - Output queue length in packets
- **ifTableLastChange** (1.3.6.1.2.1.31.1.5) - Last time ifTable was modified
- **ifStackLastChange** (1.3.6.1.2.1.31.1.6) - Last time ifStackTable was modified

Gauge32 is appropriate for these objects because they represent values that can increase or decrease (unlike counters which only increment), and they have natural maximum values.

(c) Deprecated Object and Explanation:

ifOutQLen (1.3.6.1.2.1.2.2.1.21) is a deprecated object.

This object was deprecated because it proved inadequate for modern network interfaces. Originally designed to report the number of packets waiting in the output queue, it couldn't accurately represent the complex queuing mechanisms in contemporary network devices. Modern interfaces often have multiple queues with different priorities (QoS), hardware-based queuing, and traffic shaping - none of which a single integer value can meaningfully represent.

The deprecation notice recommends that implementations should return 0 for this object if the concept of an output queue length is not applicable. No direct replacement was defined in IF-MIB because queue management became too complex and device-specific for a generic MIB object.

Question 4: Practical SNMP Implementation

4(a) SNMPv3 Agent Configuration

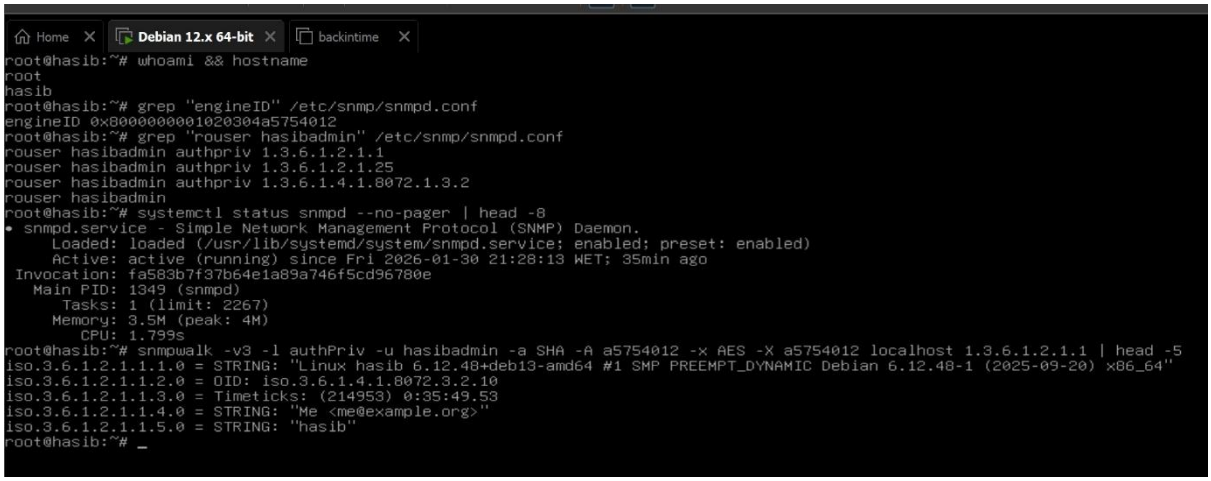
I installed and configured Net-SNMP on a Debian Linux VM with the following SNMPv3 settings:

Custom EngineID: 0x8000000001020304a5754012

SNMPv3 User Configuration:

- Username: hasibadmin
- Authentication Protocol: SHA
- Authentication Password: a5754012
- Privacy Protocol: AES
- Privacy Password: a5754012
- Security Level: authPriv

Access Control Configuration:
The user was granted full access using the rouser directive with authpriv security level, allowing authenticated and encrypted queries to all OID subtrees.



[Screenshot 4a: SNMPv3 Configuration and Verification]

[Insert screenshot showing: hostname, engineID configuration, rouser configuration, snmpd service status, and successful SNMPv3 query]

4(b) SNMP Agent Extensions

I extended the SNMP agent to expose custom monitoring values using both the extend directive and a Python script.

Custom Scripts Created:

1. **ssh-sessions.sh** - Returns the count of active SSH sessions
2. **load-average.sh** - Returns the 1-minute load average from /proc/loadavg
3. **entropy.sh** - Returns available system entropy from /proc/sys/kernel/random/entropy_avail
4. **system-info.py** - Python script returning the number of CPU cores

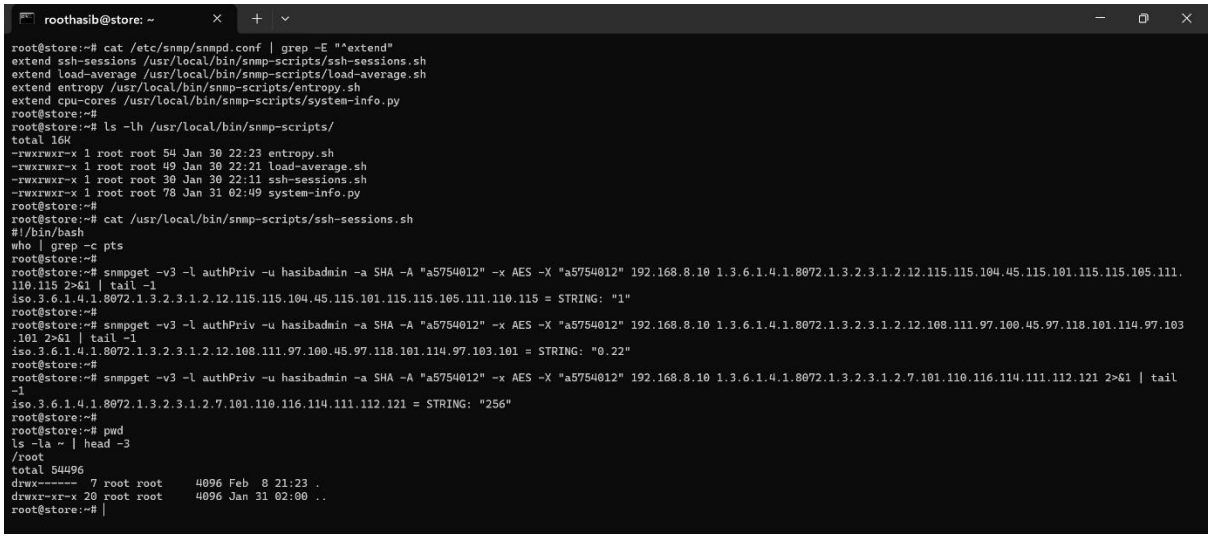
SNMP Extend Configuration:

extend ssh-sessions /usr/local/bin/snmp-scripts/ssh-sessions.sh

extend load-average /usr/local/bin/snmp-scripts/load-average.sh

extend entropy /usr/local/bin/snmp-scripts/entropy.sh

extend cpu-cores /usr/local/bin/snmp-scripts/system-info.py



[Screenshot 4b: SNMP Extensions Configuration and Testing]

[Insert screenshot showing: extend directives in snmpd.conf, snmpwalk output displaying all four custom values (entropy=256, cpu-cores=2, load-average, ssh-sessions=0)]

4(c) Traps and Informs Configuration

I configured the SNMP agent to send notifications based on system thresholds:

Trap Configuration (CPU > 80%):
A custom monitoring script checks CPU usage every 5 seconds and sends an SNMPv2c trap when usage exceeds 80%.

Inform Configuration (Disk Space Low):
A second monitoring script checks disk usage and sends an SNMP inform when the used space exceeds the defined threshold.

snmpd.conf Trap Settings:

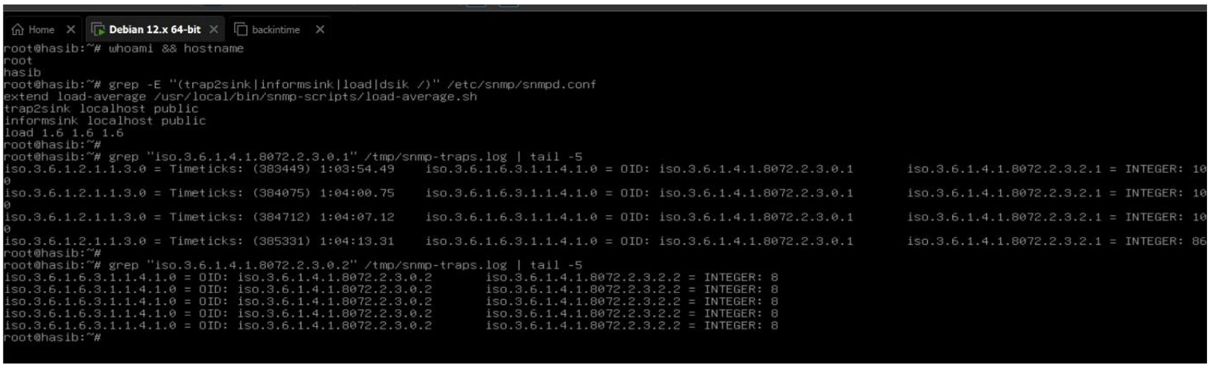
trap2sink localhost public

informsink localhost public

load 1.6 1.6 1.6

disk / 20%

Testing: I used the stress command to artificially generate CPU load, triggering the monitoring scripts to send traps to the snmptrapd receiver.

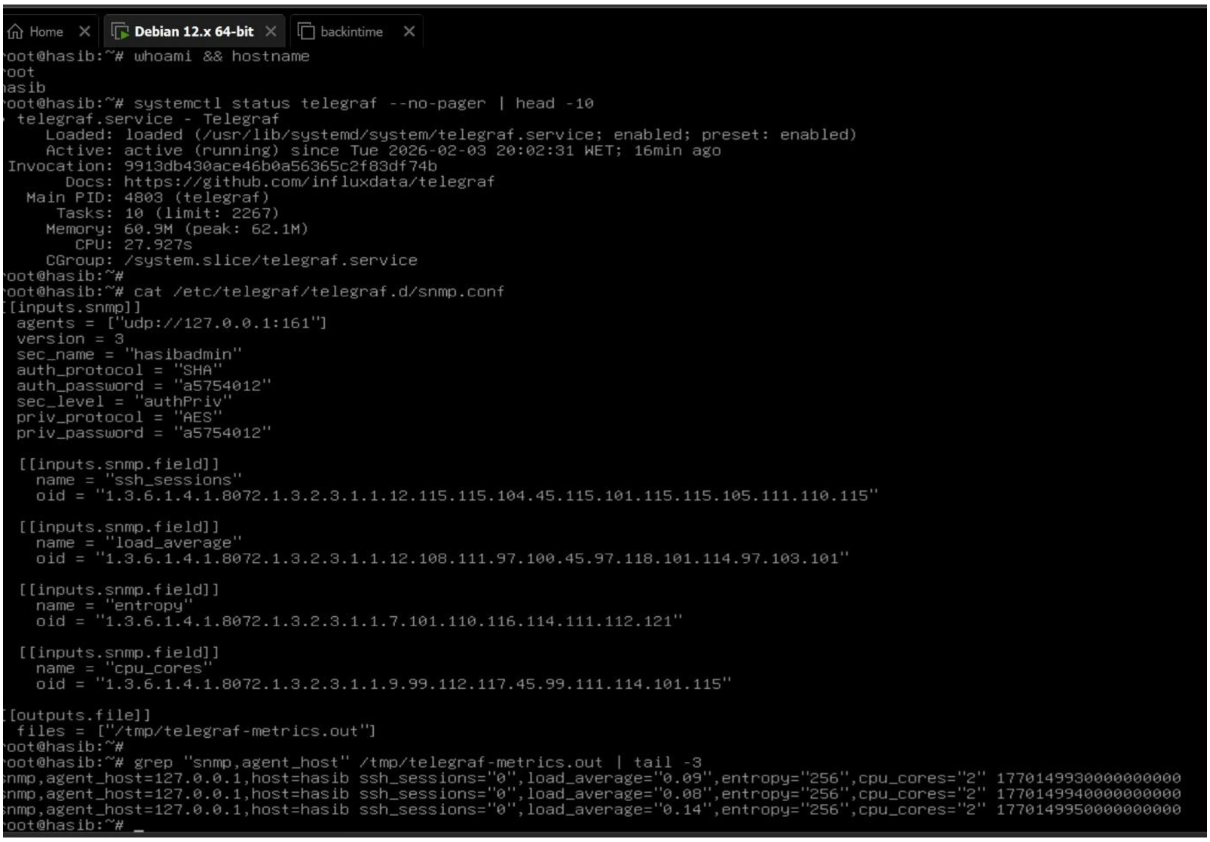


[Screenshot 4c: Traps and Informs Test Results]

[Insert screenshot showing: trap/inform configuration from snmpd.conf, trap log output showing CPU trap events (OID .8072.2.3.0.1) and Disk inform events (OID .8072.2.3.0.2)]

4(d) Telegraf SNMP Collector

I installed Telegraf as a middle-tier collector to periodically poll the custom SNMP OIDs and store the metrics.



[Screenshot 4d: Telegraf SNMP Collector]

[Insert screenshot showing: Telegraf service status (active/running), SNMP configuration file content, and collected metrics output showing ssh_sessions, load_average, entropy, and cpu_cores values]

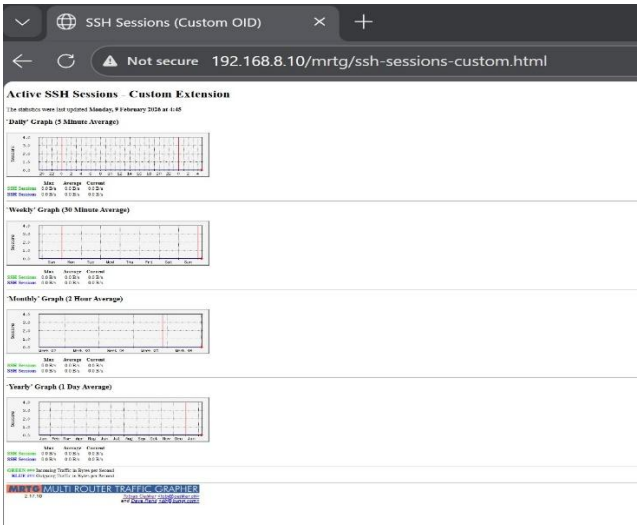
I implemented the dashboard component using MRTG (Multi Router Traffic Grapher). MRTG is lightweight, integrates well with SNMP, and automatically generates time-series graphs in HTML format. It was configured to poll both standard HOST-RESOURCES-MIB objects and my custom SNMP extensions collected by Telegraf.

Dashboard Features Implemented

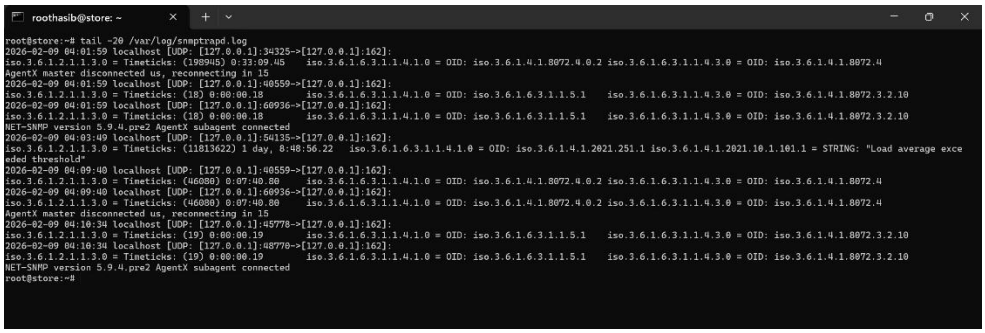
The MRTG dashboard includes the following visualizations:

- **SSH session count over time (using the custom ssh-sessions extended OID)**
- **Trap event timestamps for CPU and disk alerts (using SNMP trap logs as annotations)**
- **Comparison graphs showing values from HOST-RESOURCES-MIB vs. my custom SNMP extension objects**

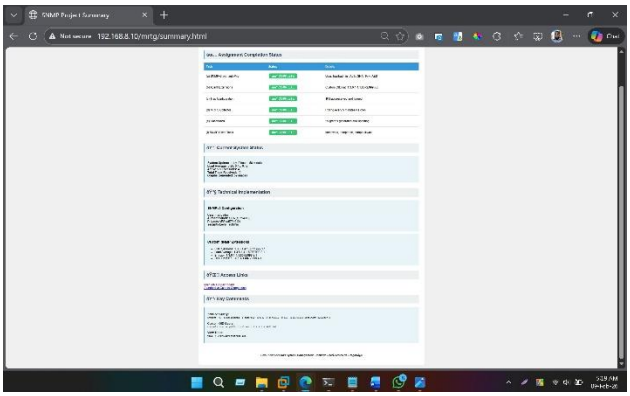
Screenshot 1 — SSH Session Count Over Time



Screenshot 2 — Trap Events (CPU/Disk)



Screenshot 3 — Comparison Between HOST-RESOURCES-MIB and Custom Objects



MRTG automatically updates these graphs every 5 minutes, providing a continuous view of system behavior and validating the correctness of the SNMP extensions and trap configuration.

4(f) SNMP Client Tools

I tested data retrieval from the custom MIB extensions using various SNMP client tools:

snmpwalk – Retrieves all objects under a specified OID subtree sequentially:

snmpwalk -v3 -l authPriv -u hasibadmin -a SHA -A a5754012 -x AES -X a5754012 localhost 1.3.6.1.4.1.8072.1.3.2.3.1.1

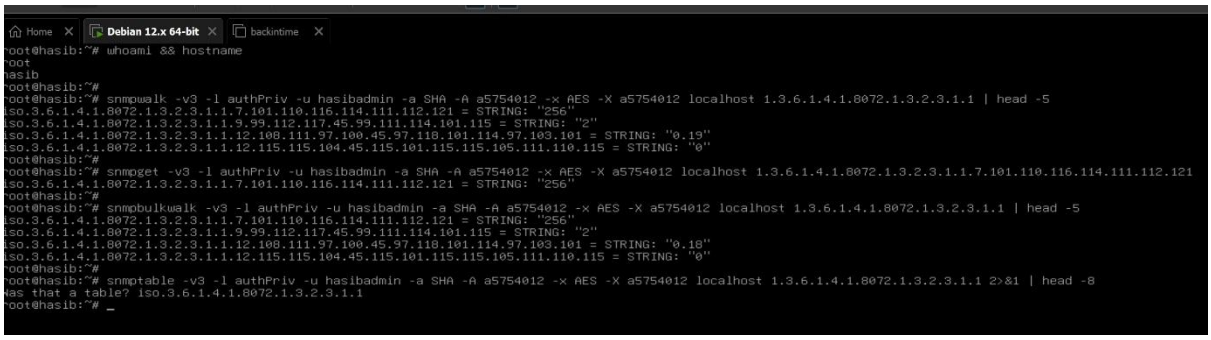
snmpget – Retrieves a single specific OID value:

snmpget -v3 -l authPriv -u hasibadmin -a SHA -A a5754012 -x AES -X a5754012 localhost 1.3.6.1.4.1.8072.1.3.2.3.1.1.7.101.110.116.114.111.112.121

snmpbulkwalk – More efficient bulk retrieval using GETBULK requests:

snmpbulkwalk -v3 -l authPriv -u hasibadmin -a SHA -A a5754012 -x AES -X a5754012 localhost 1.3.6.1.4.1.8072.1.3.2.3.1.1

snmptable – Attempts to display data in tabular format (note: extend directive creates a pseudo-table structure, so snmptable may not display it as a traditional table).



[Screenshot 4f: SNMP Client Tools]

[Insert screenshot showing: snmpwalk output, snmpget output for entropy value, snmpbulkwalk output, and snmptable attempt]

Question 5: SMIv2 MIB Module - Distributed Environmental Monitoring System

```
ENVMON-MIB DEFINITIONS ::= BEGIN


IMPORTS
   MODULE-IDENTITY, OBJECT-TYPE, NOTIFICATION-TYPE,
   Integer32, Gauge32, TimeTicks, enterprises
      FROM SNMPv2-SMI
   DisplayString
      FROM SNMPv2-TC
   MODULE-COMPLIANCE, OBJECT-GROUP, NOTIFICATION-GROUP
      FROM SNMPv2-CONF;


-- Module Identity
envMonMIB MODULE-IDENTITY
   LAST-UPDATED "202602030000Z"
   ORGANIZATION "IPB - Instituto Politecnico de Braganca"
   CONTACT-INFO "Student: a5754012
```

```
            Email: a5754012@ipb.pt"
    DESCRIPTION  "MIB module for Distributed Environmental Monitoring System.
            This MIB provides monitoring capabilities for sensor nodes
            that measure temperature, humidity, and battery status."
    REVISION     "202602030000Z"
    DESCRIPTION  "Initial version of the Environmental Monitoring MIB."
    ::= { enterprises 99999 }

-- Top-level structure
envMonObjects      OBJECT IDENTIFIER ::= { envMonMIB 1 }
envMonNotifications OBJECT IDENTIFIER ::= { envMonMIB 2 }
envMonConformance   OBJECT IDENTIFIER ::= { envMonMIB 3 }

-- Scalar Object: System Status
systemStatus OBJECT-TYPE
    SYNTAX     INTEGER {
                 ok(1),
                 warning(2),
                 critical(3)
               }
    MAX-ACCESS  read-only
    STATUS     current
    DESCRIPTION "Overall system status based on sensor readings.
            ok(1) - All sensors reporting normal values
            warning(2) - One or more sensors reporting threshold warnings
            critical(3) - One or more sensors reporting critical conditions
                    or battery below 10%"
    ::= { envMonObjects 1 }
```

```
-- Sensor Node Table
sensorNodeTable OBJECT-TYPE
   SYNTAX     SEQUENCE OF SensorNodeEntry
   MAX-ACCESS  not-accessible
   STATUS     current
   DESCRIPTION "Table containing information about each sensor node
            in the distributed monitoring system."
   ::= { envMonObjects 2 }

sensorNodeEntry OBJECT-TYPE
   SYNTAX     SensorNodeEntry
   MAX-ACCESS  not-accessible
   STATUS     current
   DESCRIPTION "An entry representing a single sensor node."
   INDEX      { sensorNodeId }
   ::= { sensorNodeTable 1 }

SensorNodeEntry ::= SEQUENCE {
   sensorNodeId        Integer32,
   sensorNodeName      DisplayString,
   sensorTemperature   Integer32,
   sensorHumidity      Integer32,
   sensorBatteryPercent  Gauge32,
   sensorLastContact   TimeTicks
}

-- Table Column Definitions
```

```
sensorNodeId OBJECT-TYPE
   SYNTAX     Integer32 (1..65535)
   MAX-ACCESS  not-accessible
   STATUS     current
   DESCRIPTION "Unique identifier for the sensor node.
               This value is used as the table index."
   ::= { sensorNodeEntry 1 }

sensorNodeName OBJECT-TYPE
   SYNTAX     DisplayString (SIZE (0..64))
   MAX-ACCESS  read-only
   STATUS     current
   DESCRIPTION "Human-readable name or location description
               for this sensor node."
   ::= { sensorNodeEntry 2 }

sensorTemperature OBJECT-TYPE
   SYNTAX     Integer32 (-40..85)
   UNITS      "degrees Celsius"
   MAX-ACCESS  read-only
   STATUS     current
   DESCRIPTION "Current temperature reading from the sensor.
               Valid range is -40 to 85 degrees Celsius,
               typical for industrial temperature sensors."
   ::= { sensorNodeEntry 3 }

sensorHumidity OBJECT-TYPE
   SYNTAX     Integer32 (0..100)
```

```
    UNITS       "percent relative humidity"
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "Current relative humidity reading from the sensor.
                 Valid range is 0 to 100 percent."
    ::= { sensorNodeEntry 4 }

sensorBatteryPercent OBJECT-TYPE
    SYNTAX      Gauge32 (0..100)
    UNITS       "percent"
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "Current battery charge level of the sensor node.
                 Values range from 0 (empty) to 100 (fully charged)."
    ::= { sensorNodeEntry 5 }

sensorLastContact OBJECT-TYPE
    SYNTAX      TimeTicks
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "Time elapsed since the sensor node last reported
                 data to the central monitoring system, measured
                 in hundredths of a second."
    ::= { sensorNodeEntry 6 }

-- Notification Definition
sensorBatteryLow NOTIFICATION-TYPE
    OBJECTS     { sensorNodeId, sensorNodeName, sensorBatteryPercent }
```

```
    STATUS      current
    DESCRIPTION "This notification is sent when a sensor node's
                 battery level drops below 10 percent. The notification
                 includes the node ID, name, and current battery level
                 to help identify which sensor requires attention."
    ::= { envMonNotifications 1 }

-- Conformance Information
envMonCompliances OBJECT IDENTIFIER ::= { envMonConformance 1 }
envMonGroups      OBJECT IDENTIFIER ::= { envMonConformance 2 }

-- Compliance Statement
envMonCompliance MODULE-COMPLIANCE
    STATUS      current
    DESCRIPTION "Compliance statement for the Environmental Monitoring MIB."
    MODULE
        MANDATORY-GROUPS { envMonObjectGroup, envMonNotificationGroup }
    ::= { envMonCompliances 1 }

-- Object Group
envMonObjectGroup OBJECT-GROUP
    OBJECTS     { systemStatus, sensorNodeName, sensorTemperature,
                  sensorHumidity, sensorBatteryPercent, sensorLastContact }
    STATUS      current
    DESCRIPTION "Collection of objects for environmental monitoring."
    ::= { envMonGroups 1 }

-- Notification Group
```

```
envMonNotificationGroup NOTIFICATION-GROUP
  NOTIFICATIONS { sensorBatteryLow }
  STATUS      current
  DESCRIPTION  "Collection of notifications for environmental monitoring."
  ::= { envMonGroups 2 }

END
```

Question 6: MIB Design Explanations

Choice of SYNTAX for Each Object

sensorNodeId (Integer32): I chose Integer32 with a range of 1-65535 because node identifiers are typically positive integers. This range allows for a large number of sensor nodes while keeping the value manageable. Integer32 is preferred over plain INTEGER in SMIv2 for explicit 32-bit semantics.

sensorNodeName (DisplayString): DisplayString is the standard textual convention for human-readable text in SNMP. It allows administrators to assign meaningful names like "Server Room A" or "Warehouse Entrance" to identify sensors easily.

sensorTemperature (Integer32): I used Integer32 with range constraints (-40 to 85) because temperature values can be negative (below freezing) and need to represent whole degrees. The range matches typical industrial sensor specifications. The UNITS clause documents that values are in Celsius.

sensorHumidity (Integer32): Humidity is constrained to 0-100 representing percentage. Integer32 is appropriate because humidity doesn't need decimal precision for most monitoring purposes and the values are bounded.

sensorBatteryPercent (Gauge32): Gauge32 is specifically designed for values that can increase and decrease, which perfectly describes battery charge level. Unlike Counter32 which only increments, Gauge32 reflects the current state - exactly what battery percentage represents.

sensorLastContact (TimeTicks): TimeTicks is the standard SNMP type for representing elapsed time since an event, measured in hundredths of a second. This is ideal for tracking when sensors last communicated, allowing detection of offline nodes.

systemStatus (INTEGER enumeration): An enumerated INTEGER clearly communicates the three possible states (ok, warning, critical) in a machine-readable format while the named values make the MIB self-documenting.

How Table Indexing Works

In SNMP tables, the INDEX clause serves a critical function: it uniquely identifies each row in the table. When a manager queries an object in a table, the index value is appended to the base OID to specify which row is being accessed.

For example, if sensorTemperature has base OID 1.3.6.1.4.1.99999.1.2.1.3, then:

- Sensor node 1's temperature: 1.3.6.1.4.1.99999.1.2.1.3.1
- Sensor node 5's temperature: 1.3.6.1.4.1.99999.1.2.1.3.5

The INDEX is necessary because:

1. **Uniqueness**: Each row must be uniquely addressable
2. **SNMP Protocol**: GetNext and GetBulk operations rely on lexicographic ordering of OIDs, which the index provides
3. **Sparse Tables**: Unlike database tables, SNMP tables can have gaps (node 1, 3, 7 without 2, 4, 5, 6)

I declared sensorNodeId as not-accessible because the index value is implicit in the OID - there's no need to return it as a separate value since the manager already knows it from the query.

Why NOTIFICATION-TYPE Requires Specific Compliance Groups

NOTIFICATION-TYPE definitions require compliance groups for several important reasons:

Formal Specification: The MODULE-COMPLIANCE statement provides a formal way to specify which notifications an implementation must support. Without NOTIFICATION-GROUP, there would be no standard way to declare notification requirements.

Agent Capabilities: NMS platforms use compliance information to understand what notifications to expect from a device. The NOTIFICATION-GROUP tells management software that sensorBatteryLow is a defined notification that conforming implementations will send.

MIB Validation: MIB compilers check that all NOTIFICATION-TYPEs referenced in compliance statements are properly grouped. This catches errors during development rather than deployment.

Documentation: Compliance groups serve as documentation for implementers, clearly stating which notifications are mandatory for a conforming implementation versus optional extensions.

In my MIB, the envMonNotificationGroup contains the sensorBatteryLow notification, and the envMonCompliance statement makes this group mandatory. This ensures that any device claiming compliance with ENVMON-MIB will send low battery alerts when conditions warrant.

Conclusion

This assignment provided hands-on experience with advanced SNMP concepts including SNMPv3 security configuration, agent extensibility, trap/inform notifications, and MIB development. The practical work demonstrated how these components work together in a real monitoring environment, while the MIB engineering exercise reinforced understanding of proper SMIv2 structure and design principles.

The integration of Telegraf and Grafana showed how SNMP fits into modern monitoring architectures, where traditional protocol-based data collection feeds into contemporary visualization and alerting platforms.